Allocation for Omnidirectional Aerial Robots: Incorporating Power Dynamics

Eugenio Cuniato, *Member, IEEE,* Mike Allenspach, *Member, IEEE,* Thomas Stastny, *Member, IEEE,* Helen Oleynikova, *Member, IEEE,* Roland Siegwart, *Fellow, IEEE,* Michael Pantic, *Member, IEEE*

***Abstract*—Tilt-rotor aerial robots are more dynamic and versatile than fixed-rotor platforms, since the thrust vector and body orientation are decoupled. However, the coordination of servos and propellers (the *allocation* problem) is not trivial, especially accounting for overactuation and actuator dynamics. We incrementally build and present three novel allocation methods for tilt-rotor aerial robots, comparing them to state-of-the-art methods on a real system performing dynamic maneuvers. We extend the state-of-the-art geometric allocation into a differential allocation, which uses the platform's redundancy and does not suffer from singularities. We expand it by incorporating actuator dynamics and propeller *power dynamics*. These allow us to model dynamic propeller acceleration limits, bringing two main advantages: balancing propeller speed without the need of nullspace goals and allowing the platform to selectively turn-off propellers during flight, opening the door to new manipulation possibilities. We also use actuator dynamics and limits to normalize the allocation problem, making it easier to tune and allowing it to track 70% faster trajectories than a geometric allocation.**

***Index Terms*—Aerial Systems: Mechanics and Control, Direct/Inverse Dynamics Formulation, Motion Control, Omnidirectional Aerial Robots**

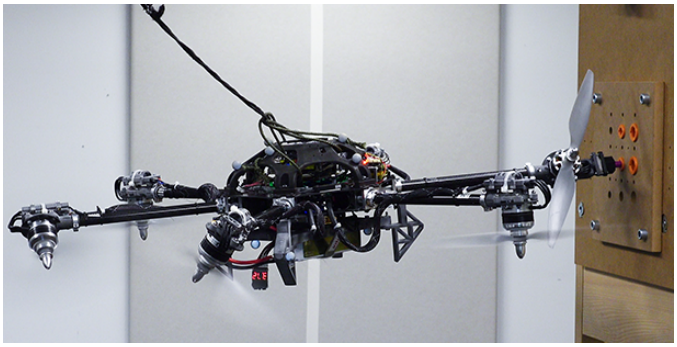

Fig. 1: An omnidirectional tilt-rotor aerial robot screwing a bolt into a wall. The robot can smoothly transition its arms from flying mode (propellers on) to interaction (propellers off) with our novel allocation method. Video: youtu.be/ybT80MyB9_Y

I. Introduction

Omnidirectional aerial robots have enjoyed increasing interest in the aerial robotics community [1]. The actuation capabilities of these systems allow omnidirectional thrust generation, resulting in fully-decoupled translational and rotational dynamics. Recent works on aerial physical interaction demonstrate the superiority of omnidirectional platforms over traditional underactuated ones, ensuring precise motion and interaction force control with simultaneous disturbance rejection [2]. Depending on the specifics of the actuation, omnidirectional aerial robots can be classified as either *fixed-rotor* or *tilt-rotor* platforms. Fixed-rotor systems feature propellers rigidly mounted at specific angles [3]–[6]. While mechanically simple, they impose a fixed trade-off between efficiency and interaction capability, with no adaptability to mission or flight context.

Instead, tilt-rotor vehicles make use of dedicated actuators to modify the propeller orientation, depending on the task at hand [7]–[10]. This versatility strongly promotes their use in aerial interaction applications, where transitioning between an efficient navigation and a high-force interaction configuration might be required at different points throughout the mission [2], [11]. However, it requires elaborate control allocation schemes to handle potential singularities, resolve actuator redundancy, and respect the difference in dynamic response between the tilt motors and the propellers.

These challenges have traditionally been addressed using complex, computationally expensive optimizers [9], [12] or through heuristics in geometric allocation schemes [13], [14]. More recently, differential control allocation has emerged as a promising alternative [8], [15]. By formulating the allocation problem at the differential level, this approach incorporates not only geometric information but also the system's kinematics. However, despite its potential, significant gaps remain in the literature, as outlined in Section II: (i) performance often depends on high-quality 6-DOF acceleration measurements; (ii) actuator dynamics and limits are usually neglected or oversimplified; (iii) propeller speed balancing for energy efficiency typically relies on complex nullspace objective switching; and (iv) thorough comparisons between differential and geometric allocation methods are still missing.

Aiming to address these limitations and enhance tilt-rotor robot capabilities, we propose and discuss different control allocation methodologies, experimentally compared on the platform shown in Fig. 1. Our focus is on embedding actuator dynamics and limits directly into the allocation, leveraging overactuation without nullspace objectives, thereby enabling smooth in-flight propeller deactivation, opening new avenues for crash resilience and aerial manipulation.

All authors are with the Autonomous Systems Lab (ASL), ETH Zurich. Corresponding author: ecuniato@ethz.ch.

The research leading to these results has been supported by the AERO-TRAIN project, European Union's Horizon 2020 research and innovation program under the Marie Skłodowska-Curie grant agreement No 953454, the ETH RobotX Research Program and the Armasuisse Research Grant No 3200000100. The authors are solely responsible for its content.

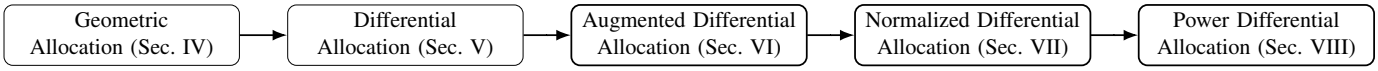

Fig. 2: The allocation methods introduced in this paper. The Geometric Allocation (GA) and Differential Allocation (DA) are the state-of-the-art geometric and differential allocation methods, respectively. The Augmented Differential Allocation (ADA) is a novel differential allocation method that does not require acceleration feedback. The Normalized Differential Allocation (NDA) normalizes the allocation problem with the actuator dynamics and limits, while the Power Differential Allocation (PDA) introduces propeller power dynamics to minimize the use of nullspace commands and allow propeller deactivation in flight.

A. Contributions

We incrementally address the challenges outlined above through the development of three novel actuator allocation schemes for tilt-rotor platforms:

1) We achieve up to 70% faster trajectory tracking than geometric allocation by eliminating the need for acceleration feedback in differential allocation, enabling seamless integration with standard wrench-based control architectures.
2) We make the allocation aware of its actuators limits and dynamics, balancing the utilization of our diverse actuators (propeller and servomotors) without requiring the manual tuning of a weighted pseudoinverse.
3) We embed propeller power dynamics and model acceleration limits into the allocation description, providing a powerful new tool for optimizing flight efficiency and desired propeller speeds, without requiring elaborate nullspace objective switching.
4) We demonstrate how these curves can be used to turn off propellers in flight, allowing the platform to control its propelling arms as manipulation tools, removing the need for additional manipulators.

We begin the paper with a more in-depth review of related work in control allocation for tilt-rotor aerial robots in Section II, further detailing the limitations outlined above. The rest of the paper is organized as shown in Fig. 2, first introducing the overall system architecture in Section III, and then discussing and incrementally building up the five compared allocation methods (two state-of-the-art, three novel). We then do a quantitative experimental comparison of all methods in Section IX and show a qualitative demonstration of the ability of our proposed allocation to stop propellers in-flight for manipulation in Section IX-D.

II. Related Work

Considering their primary use-case as aerial manipulators, modern tilt-rotor aerial robots are mechanically designed to optimize omnidirectional force/torque envelopes, while exhibiting a dominant hover orientation for efficient flight. As shown in [8], this optima is achieved by following a standard multicopter morphology, meaning all propellers are mounted in a co-planar and symmetric fashion on the main body. Unlike standard underactuated multicopters, the direction of each propeller axis can be adjusted in flight, due to actuated tilting joints at the mounting point of the arms. To achieve a desired force and torque acting on the body, we need to solve the *allocation problem* of choosing the best distribution of propeller speeds and tilt arm directions across all the actuators. In the following we summarize common solutions in the literature of tri- [7], quad- [10], [15]–[17] and hexa-copter-like [2], [8], [9], [12], [18]–[21] platforms.

A. Hierarchical Allocation

Motivated by the comparatively slow dynamics of the tilt arms, [18] proposes to optimize tilt angles along a given trajectory for efficient flight before the start of each mission, but then keeping them constant during execution. To still ensure accurate tracking in the presence of disturbances, only the propeller speeds are updated online using a well-known matrix-inversion-based allocation scheme [22]. Aside from requiring knowledge of the full task trajectory ahead of time, naively optimizing tilt angles for efficiency can result in singularities and loss of omnidirectionality [20]. Follow-up works [9], [12], [19] address this limitation by proposing to use an online tilt angle planner instead. While also separately allocating tilt angles and rotor speeds, such a planner would aim to prevent singularities, without introducing excessive internal forces that reduce energy efficiency. However, the choices of tilt angle in different flight situations seem to be very much task dependent and heuristically defined by the user. In short, incorporating control authority and efficient flight while avoiding singularities is not trivial when tilt angle and propeller speed allocation are performed separately.

B. Unified Geometric Allocation

Alternatively, control allocation of both tilt angles and rotor speeds can be combined, resulting in a nonlinear and potentially high-dimensional system of equations to be solved. Approaches presented in [7], [13], [14], [16], [21], [23] demonstrate how propeller force decomposition and trigonometric identities can produce a linear formulation suitable for matrix-inversion-based *geometric allocation* schemes (see Sec IV). Since the matrix becomes ill-conditioned when the system approaches a singular configuration, singularity cases must be analyzed and handled carefully. Possible heuristics are proposed and experimentally validated in [21], for example biasing the desired arm angles to avoid numerical singularities. However, the mathematical problem of mapping a desired wrench to tilt angles and rotor speeds still intrinsically suffers from singularities.

It should be noted that the cited works often neglect the *different* dynamics between propellers and servomotors [2], which can result in degraded disturbance rejection and dynamic tracking (see Sec. IX).

C. Unified Differential Allocation

To better coordinate the tilting motion and propeller dynamics, recent works propose to control the tilt angle *velocities* and rotor *accelerations* to recreate a desired change in total body force and torque, rather than forces and torques themselves. In other words, to move the allocation problem to a higher differential level [8], [15]. This so-called *differential allocation* (see Sec V) is not only more realistic in terms of actuator dynamics (as we no longer assume an instantaneous change of tilt angles and rotor speeds), but is also inherently robust to singularities [8]. Furthermore, resolution of the actuator redundancy through nullspace exploitation is straightforward, for example to improve efficiency [15] or for consideration of mechanical constraints [8].

While promising, existing differential allocation methods require measuring translational and rotational body accelerations. Although acceleration feedback has been achieved on racing drones using onboard sensors [24], aerial manipulation platforms typically operate at much lower speeds and accelerations ($20\,\mathrm{m/s}$, $49\mathrm{m/s^2}$ vs. $1\,\mathrm{m/s}$, $2\,\mathrm{m/s^2}$ [8], [13], [15]). These gentler but precise maneuvers, combined with omnidirectional designs that decouple translational and rotational motion, result in minimal IMU excitation. Consequently, the signal-to-noise ratio is significantly reduced, making it difficult to obtain reliable acceleration measurements [23]. Additionally, compared to racing drones, propeller speeds on aerial manipulation platforms tend to be more variable and operate closer to the system's natural frequencies (e.g., $22\,000\,\mathrm{RPM} \equiv 366\,\mathrm{Hz}$ vs. $5000\,\mathrm{RPM} \equiv 83\,\mathrm{Hz}$ [8], [9], [15]), which complicates effective filtering.

Moreover, state-of-the-art differential allocation methods always require the use of posture nullspace objectives even just for free-flight, which keeps the nullspace always full and requires task switching in case a change in the nullspace goals is needed [25] (e.g. from efficiency to mechanical constraints or manipulation tasks).

In this work, we iteratively build up three novel actuator allocation schemes for tilt-rotor platforms that integrate actuator dynamics, constraints, and potential secondary objectives into a unified differential allocation framework. Unlike existing approaches, our methods do not rely on acceleration feedback and minimizes the use of nullspace objective switching by directly encoding all relevant properties into the primary allocation problem. We provide a quantitative experimental comparison against state-of-the-art geometric [21] and differential [8] allocation strategies, demonstrating superior stability and dynamic tracking performance.

III. System architecture

Tilt-rotor aerial robots have a number of propeller arms which can independently rotate around their axis using dedicated servomotors. Each arm mounts a rotor at its tip, creating an independent thrust unit. With three rotating propelling arms, the platform is fully-actuated, decoupling position and orientation control in space. With more arms, it becomes overactuated, adding a nullspace that can be used to optimize further metrics apart from the main flight task.

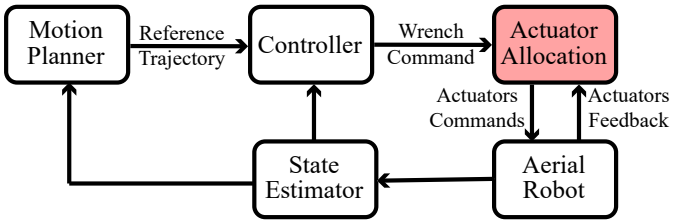

Fig. 3: A general control architecture for tilt-rotor aerial robots. The pose controller compares the current odometry to a reference trajectory and produces a wrench command. The wrench command is finally allocated to the actuators of the robot, namely servomotors and propellers, to generate the desired wrench which drives the platform. In this work we focus on the allocation block, highlighted in red.

For these robots, a common control architecture is shown in Fig. 3. Given a set of sensors available on aerial robots (GPS, IMU, LiDAR, optical flow, ...), the state estimator provides an odometry measure [26], which the controller [2] uses to generate a command wrench (forces and torques) to follow the current position and/or velocity reference. Finally, the command wrench $\boldsymbol{w} \in \mathbb{R}^6$ is transformed into actuator commands for the platform's arm angles $\boldsymbol{\alpha} = [\alpha_0, ..., \alpha_{N-1}]^\top$ and propeller speeds $\boldsymbol{\omega} = [\omega_0, ..., \omega_{N-1}]^\top$ through the chosen allocation method, with $N \in \mathbb{Z}^+$ the number of arms.

In the following sections, we keep the motion planner, controller and state estimator fixed as in [2], and focus on the actuator allocation problem. A schematic of the allocation methods that will be introduced in the following sections can be seen in Fig. 2.

IV. Geometric Allocation (GA)

For the reader's convenience, here we reproduce the state-of-the-art geometric allocation method of [21], which only uses the platform's geometry to map the desired control wrench into propeller speeds and arm rotation angles. [21] defines the allocation problem as

$$\boldsymbol{w} = \boldsymbol{A}\boldsymbol{u}\ ,\ \boldsymbol{u} = \begin{bmatrix} \omega_0 \sin\alpha_0 \\ \omega_0 \cos\alpha_0 \\ \dots \\ \omega_{N-1} \sin\alpha_{N-1} \\ \omega_{N-1} \cos\alpha_{N-1} \end{bmatrix}, \tag{1}$$

where $\boldsymbol{A} \in \mathbb{R}^{6\times 2N}$ represents a constant allocation matrix depending only on the system's geometry, while $\boldsymbol{u} \in \mathbb{R}^{2N}$ is the result of the allocation, obtained pseudoinverting the matrix $\boldsymbol{A}$. Once the command vector $\boldsymbol{u}$ is generated, we extract the actual rotor speeds and tilt angles with

$$\alpha_i = \mathrm{atan2}\left(u_{2i}, u_{2i+1}\right)\ ,\ \forall i = 0 \dots N-1 \tag{2a}$$

$$\omega_i = \sqrt{u_{2i}^2 + u_{2i+1}^2}\ ,\ \forall i = 0 \dots N-1. \tag{2b}$$

This method, which takes inspiration from the allocation of standard underactuated aerial robots, has the advantage of only requiring knowledge of the system's geometry. However, it has a few limitations:

- Since the allocation is purely geometric, it does not account for actuator dynamics or limits.
- The matrix $\boldsymbol{A}$ is constant and full rank, but the allocation problem can become singular depending on the desired wrench [21], i.e., it's not always possible to extract angles $\boldsymbol{\alpha}$ and rotor speeds $\boldsymbol{\omega}$ with Eq. (2).
- The geometric allocation does not exploit the system's redundancy, only providing the solution with the minimum rotor speeds.

To overcome some of these limitations, [8] introduced a differential allocation method.

V. Differential Allocation (DA)

The idea is to differentiate the relation in Eq. (1) to map the time derivative of the wrench command (which we will from now on refer to as *jerk*) to arm tilting speed and propeller acceleration, as

$$\dot{\boldsymbol{w}} = \boldsymbol{A}\boldsymbol{D}(\boldsymbol{q})\dot{\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}, \tag{3}$$

where $\dot{\boldsymbol{w}} \in \mathbb{R}^6$ is the desired jerk to apply on the robot's body, $\boldsymbol{q} = \left[\boldsymbol{\alpha}^\top, \boldsymbol{\omega}^\top\right]^\top \in \mathbb{R}^{2n}$ is the vector of joint states and $\boldsymbol{D} = \frac{\partial \boldsymbol{u}}{\partial \boldsymbol{q}} \in \mathbb{R}^{2N \times 2N}$ is the Jacobian of the geometric actuation vector $\boldsymbol{u}$.

The solution to this allocation problem is then

$$\dot{\boldsymbol{q}} = \boldsymbol{J}^\ddagger \dot{\boldsymbol{w}} + (\boldsymbol{I}_{2N} - \boldsymbol{J}^\ddagger \boldsymbol{J})\dot{\boldsymbol{q}}^*, \tag{4}$$

where the allocation's output is now a vector $\dot{\boldsymbol{q}} \in \mathbb{R}^{2N}$ of arm rotation speeds and rotor accelerations, $\dot{\boldsymbol{q}}^* \in \mathbb{R}^{2N}$ is a vector of desired joint velocities to exploit the system's overactuation by fulfilling secondary goals.

While distributing the desired jerk among the actuators, we must balance the usage of tilt angles and propellers, to avoid pushing either into saturation. In [8], the authors propose a weighted pseudoinverse

$$\boldsymbol{J}^\ddagger = \boldsymbol{W}^{-1}\boldsymbol{J}^\top \left(\boldsymbol{J}\boldsymbol{W}^{-1}\boldsymbol{J}^\top\right)^{-1}, \tag{5}$$

with a positive-definite weight matrix $\boldsymbol{W} \in \mathbb{R}^{2N \times 2N}$ to distribute the desired jerk among the tilting arms and propellers.

This allocation method solves the singularity problem of the geometric solution in Section IV [8], while enabling use of the system's overactuation with $\dot{\boldsymbol{q}}^*$. A possible choice for $\dot{\boldsymbol{q}}^*$ is to drive the propellers' speeds to a desired value (for example the hovering speed ω^*) with

$$\dot{\boldsymbol{q}}^* = -k_\omega \left[\boldsymbol{0}_6, (\omega_0 - \omega^*), ..., (\omega_{N-1} - \omega^*)\right]^T, \tag{6}$$

where $k_\omega \in \mathbb{R}$ is a positive gain. Other choices, like actuator limits avoidance [15], can be found in the literature of robot manipulators [27].

However, this method has two key limitations: it allocates a jerk command and relies on a heuristically chosen weighted pseudoinverse $\boldsymbol{W}$ to balance arm and propeller use. Both points are discussed in more detail in the following two subsections.

A. Jerk vs. Wrench Control

While the differential allocation requires a jerk command, wrench control is still the most common solution for aerial robots. Indeed, it is the natural solution to the second-order rigid body dynamics, and it only requires position and velocity feedback, both commonly available on aerial robots.

The authors in [8] adopted a Linear Quadratic Regulator (LQR) to produce the control jerk from acceleration feedback. They designed two filters to obtain linear and angular acceleration feedback by filtering and differentiating Inertial Measurement Unit (IMU) data.

This solution permits the use of a differential allocation method, but its trajectory tracking performance shows no improvement over a standard PD controller with geometric allocation [8]. We attribute this inefficiency to the LQR's strong model dependency and the limited quality of acceleration feedback.

As discussed in Section II, low-excitation maneuvers reduce the IMU's signal-to-noise ratio, complicating both filtering and differentiation of acceleration signals.
For this reason, in the following Section VI we propose an augmented differential allocation, which does not require acceleration feedback and can be easily implemented with any standard wrench-based controller.

B. Weighted Pseudoinverse

Differential allocation methods are common, for example, in the context of arm manipulators, where the behavior of the several actuators is similar and the joint speeds are usually in the same range. However, in the case of tilt-rotor aerial robots, the behavior of the actuators is very different, as $\dot{\boldsymbol{q}}$ contains both propeller acceleration and rotation speed of the arms. These two, apart from having different physical units and meanings, also have very different ranges, with the propeller accelerations usually around $\dot{\omega} \sim 10^4 RPM/s$ and the arm speeds around $\dot{\alpha} \sim 1 rad/s$.

To keep the problem numerically stable, the differential allocation relies on a weighted pseudoinverse, which requires manual tuning of the weight matrix $\boldsymbol{W}$ to balance the usage between tilt angles and propellers. The matrix $\boldsymbol{W}$ has a prominent effect on the aerial robot's behavior: too much weight on the tilt angles will push the allocation to use only the propellers, behaving the same way as a fixed multirotor, whereas too much weight on the propellers can lead the tilt angles' servomotors to rotate very fast and quickly saturate. Finding a reliable $\boldsymbol{W}$ often requires hours of in-flight tuning.

In Section VII, we propose a method to integrate actuator limits into the allocation problem to improve its numerical conditioning without the need of a hand-tuned weight matrix.

VI. Augmented Differential Allocation (ADA)

Here we propose Augmented Differential Allocation, to bring all the advantages of the Differential Allocation in Eq. (4) to any standard wrench-based controller, without the need for acceleration feedback. We augment the input

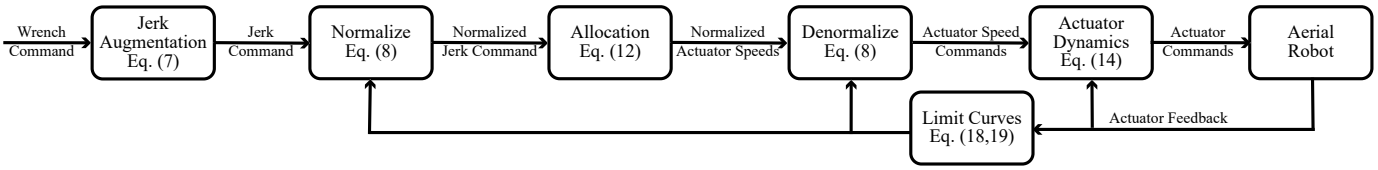

Fig. 4: The proposed differential allocation scheme. We first augment the wrench command to generate a jerk command, we then normalize it and allocate into numerically stable normalized joint speeds. Finally, we invert the actuator dynamics to obtain the desired tilt angles and propeller speeds to command the Aerial Robot.

dynamics of our system in order to generate a jerk control command from a wrench command as

$$\dot{\boldsymbol{w}} = k_j \left(\boldsymbol{w}^d - \boldsymbol{w}(\boldsymbol{q}) \right), \tag{7}$$

where $k_j \in \mathbb{R}$ is a gain generating a jerk proportional to the error between the current wrench desired command $\boldsymbol{w}^d$ coming from a higher-level controller and the current wrench applied on the robot $\boldsymbol{w}(\boldsymbol{q})$ computed from the current actuators' state feedback. This approach is much more reliable than IMU feedback and differentiation: on one hand, it does not rely on platform- or sensor-dependent IMU filtering; on the other, the actuators inherently act as low-pass filters, naturally providing a less noisy estimate.

Moreover, this jerk augmentation was already validated in the context of Control Barrier Functions (CBF) for ground [28] and aerial robots [29]. However, it still requires tuning of the weight matrix $\boldsymbol{W}$ in Eq. (5) to balance the usage between tilt angles and propellers, which ultimately still relies on user heuristics obtained in flight tests.

VII. Normalized Differential Allocation (NDA)

To avoid heuristically tuning the weight matrix $\boldsymbol{W}$, we propose to balance the actuators' usage by normalizing the allocation problem with the knowledge of the actuators' control limits, instead of using a weighted pseudoinverse.

A. Actuator Control Limits

To normalize the allocation problem, consider the minimum and maximum actuator rate vectors $\underline{\dot{\boldsymbol{q}}} = \left[\underline{\dot{\boldsymbol{\alpha}}}^\top, \underline{\dot{\boldsymbol{\omega}}}^\top\right]^\top$ and $\overline{\dot{\boldsymbol{q}}} = \left[\overline{\dot{\boldsymbol{\alpha}}}^\top, \overline{\dot{\boldsymbol{\omega}}}^\top\right]^\top$. We refer with *actuator rate* $\dot{\boldsymbol{q}}$ to angular velocities for the tilt angles and rotor accelerations for the propellers. This is because the state of our actuators $\boldsymbol{q}$ is composed of both tilt angles and rotor speeds, which can be regarded as an *actuator position*.

We now normalize the actuator velocities with respect to these limits, mapping $\dot{q}_i \in [\underline{\dot{q}}_i, \overline{\dot{q}}_i]$ to $\dot{q}_{\mathrm{n},i} \in [-1, 1]$ with

$$\dot{\boldsymbol{q}}_\mathrm{n} = \boldsymbol{N}\dot{\boldsymbol{q}} - \boldsymbol{b}, \tag{8}$$

with a diagonal scaling matrix $\boldsymbol{N} \in \mathbb{R}^{2N \times 2N}$ and a bias vector $\boldsymbol{b} \in \mathbb{R}^{2N}$, where

$$N_{i,i} = \frac{2}{\overline{\dot{q}}_i - \underline{\dot{q}}_i}, \; b_i = \frac{\overline{\dot{q}}_i + \underline{\dot{q}}_i}{\overline{\dot{q}}_i - \underline{\dot{q}}_i}, \; \forall i = 0, \ldots, 2N-1. \tag{9}$$

We now rewrite the allocation problem in Eq. (3) as a function of the normalized joint speeds $\dot{\boldsymbol{q}}_\mathrm{n}$ instead of the speeds themselves $\dot{\boldsymbol{q}}$. Substituting Eq. (8) in the original differential allocation Eq. (3) we obtain the normalized allocation

$$\dot{\boldsymbol{w}} = \boldsymbol{J}(\boldsymbol{q})\boldsymbol{N}^{-1}\dot{\boldsymbol{q}}_\mathrm{n} + \boldsymbol{J}(\boldsymbol{q})\boldsymbol{N}^{-1}\boldsymbol{b}, \tag{10}$$

which is equivalent to

$$\dot{\boldsymbol{w}}_\mathrm{n} = \boldsymbol{J}_\mathrm{n}\dot{\boldsymbol{q}}_\mathrm{n}, \tag{11}$$

where the normalized jerk command to allocate is now $\dot{\boldsymbol{w}}_\mathrm{n} = \dot{\boldsymbol{w}} - \boldsymbol{J}\boldsymbol{N}^{-1}\boldsymbol{b} \in \mathbb{R}^6$ and the normalized allocation matrix is $\boldsymbol{J}_\mathrm{n} = \boldsymbol{J}\boldsymbol{N}^{-1} \in \mathbb{R}^{2N \times 2N}$. Similarly to the original differential allocation, the solution to this problem is

$$\dot{\boldsymbol{q}}_\mathrm{n} = \boldsymbol{J}_\mathrm{n}^\dagger \dot{\boldsymbol{w}}_\mathrm{n} + (\boldsymbol{I}_{2N} - \boldsymbol{J}_\mathrm{n}^\dagger \boldsymbol{J}_\mathrm{n})\dot{\boldsymbol{q}}_\mathrm{n}^*, \tag{12a}$$

$$\boldsymbol{J}_\mathrm{n}^\dagger = \boldsymbol{J}_\mathrm{n}^\top \left(\boldsymbol{J}_\mathrm{n}\boldsymbol{J}_\mathrm{n}^\top\right)^{-1}. \tag{12b}$$

Despite the similarity, this formulation has the advantage of not requiring manual tuning of a weighted pseudoinverse. Instead, actuator balancing is naturally induced by their rate limits, as used in Eq. (9), which we identified experimentally on a test bench.

B. Actuator Saturation

Despite normalizing the actuator commands between $[-1, 1]$, given a normalized jerk $\dot{\boldsymbol{w}}_\mathrm{n}$, the allocation might still output normalized joint velocity commands $\dot{\boldsymbol{q}}_\mathrm{n}$ outside of these boundaries. This can happen if the required jerk command is too high, for example when requiring very rapid maneuvers. We saturate the joint velocities between $[-1, 1]$, by linearly scaling down the velocity vector with a quantity $k_s < 1$ as

$$\mathrm{sat}(\dot{\boldsymbol{q}}_\mathrm{n}) = k_s \dot{\boldsymbol{q}}_\mathrm{n}. \tag{13}$$

The desired gain is $k_s = \frac{1}{|\dot{q}_{\mathrm{n},b}|}$, where $\dot{q}_{\mathrm{n},b}$ is the biggest element of the joint speeds vector. Notice that since all the actuator commands in $\dot{\boldsymbol{q}}_\mathrm{n}$ are scaled by the same quantity k_s and the relation in Eq. (11) is linear, this corresponds to generating a jerk $\mathrm{sat}(\dot{\boldsymbol{w}}_\mathrm{n})$ which has the same direction of the original $\dot{\boldsymbol{w}}_\mathrm{n}$ but scaled in magnitude. While such scaling is well established in robotic manipulator control [27], [30], it remains unique in aerial robotics. Even in the few works that adopt jerk-level allocation, actuator saturation is handled through simple clipping [8] or heuristic null-space strategies without feasibility guarantees [15]. In contrast, our method enables a more intuitive and systematic approach by leveraging proven techniques from robotic manipulators, highlighting the conceptual similarity between omnidirectional aerial robots and fully-actuated robot arms.

C. Actuator Dynamics

The described allocation procedure yields actuator rate commands $\dot{\boldsymbol{q}}$ that aim to realize the desired jerk $\dot{\boldsymbol{w}}$. While the standard approach is to integrate these rates to obtain actuator setpoints $\boldsymbol{q}$ [8], [15], we leverage the differential formulation to incorporate not only actuator velocity *limits* but also their *dynamics*.

Using data from real flights, we identify a first-order actuator model via least-squares fitting:

$$\dot{\boldsymbol{q}} = -\boldsymbol{K}\boldsymbol{q}_e, \tag{14}$$

where $\boldsymbol{K} \in \mathbb{R}^{2N\times 2N}$ is a positive-definite diagonal gain matrix, and $\boldsymbol{q}_e \in \mathbb{R}^{2N}$ represents angular and rotational speed errors for tilt arms and propellers, respectively.

This first-order model aligns naturally with our rate-based allocation. In contrast, second-order dynamics would require differentiating Eq. (12) and access to propeller acceleration feedback, typically unavailable from current Electronic Speed Controllers (ESCs).

We thus compute desired actuator rates by solving the allocation as before and then invert Eq. (14) to obtain dynamically feasible actuator setpoints. All the steps of the proposed allocation process are summarized in Fig. 4.

VIII. Power Differential Allocation (PDA)

Differential allocation controls the actuators' rate (propeller acceleration and tilt arm speed), but not directly their states (propeller speed and tilt arm angles). This might not be a problem for the tilt arms, which could virtually rotate indefinitely depending on their mechanical implementation.

However, propellers' speed are limited by the maximum power that the ESCs can provide, propeller's inertia and drag coefficient. Aside from these constraints, balanced thrust distribution is particularly important for tiltrotor aerial vehicles. At high pitch or roll angles, some propellers may spin significantly faster than others. If not *actively* rebalanced upon returning to level hover, the system can remain in a power-inefficient state, unnecessarily pushing actuators toward saturation.

The differential allocations introduced so far rely on nullspace goals such as Eq. (6) to keep the speed balanced and avoiding propellers to hit the limit. Here we introduce the propeller power dynamics: a model in which acceleration limits in Eq. (8) adapt dynamically based on current propeller speeds. This allows for both enforcing speed constraints and encouraging speed equalization directly within the allocation, without requiring an explicit nullspace objective. This contrasts with established approaches in robotic manipulation [27], [30], which commonly introduce a secondary nullspace allocation goal to optimize the robot's posture. Rather than aiming to outperform such nullspace strategies, our method preserves the nullspace for aerial-robotics-specific objectives such as internal force regulation [27], mechanical considerations [8], or manipulation-oriented tasks (see Section IX-D).

We derive the power dynamics in two steps: First, in Section VIII-A, we identify the physical limits imposed by the ESCs, propeller drag, and inertia. Then, in Section VIII-B, we propose a limit curve formulation that respects these mechanical constraints while also encouraging speed equalization.

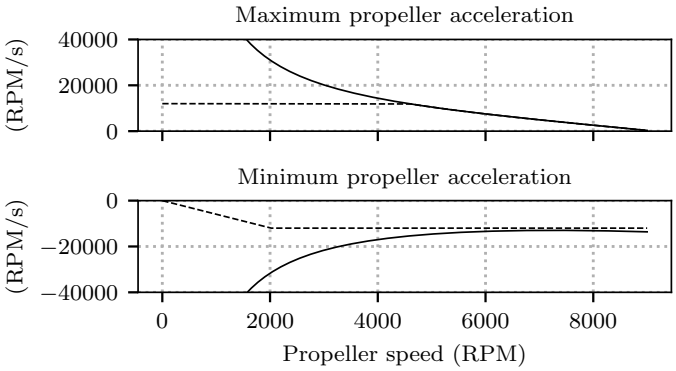

Fig. 5: Example acceleration limit curves (solid lines) from the propellers power balance in Eq. (15). The dashed lines represent the saturated versions of the curves, where the maximum and minimum accelerations have been capped to represent the internal software ESC limits. For very low propeller speeds, the drag torque has a very low effect, leading to very high acceleration and deceleration limits. As speed increases, the quadratic drag force overtakes the electrical torque, leading to a reduction in the maximum acceleration and deceleration, until the acceleration becomes zero when the propeller reaches the maximum speed. The curves were obtained with $\eta = 0.8$, $V = 23V$, $\overline{I} = -\underline{I} = 17A$, $J = 4.5e^{-4}Kgm^2$, $d = 3.5e^{-7}Nms^2$ and $\overline{\dot{\omega}} = -\underline{\dot{\omega}} = 1.3e^4 RPM/s$.

A. Real Propeller Curves

The propeller acceleration curves for a generic propeller motor can be obtained through the power equilibrium equation. Specifically, the input electrical power must be equal to the power dissipated to fight the propeller's inertia and drag torque. This translates to

$$\eta V I = J\omega\dot{\omega} + d\omega^3, \tag{15}$$

where all the quantities are scalar and $\eta < 1$ is the electrical efficiency of the motor, V is the voltage provided to the ESC, I is the current absorbed by the motor, J is the propeller and rotor combined inertia and d is the drag coefficient. If we consider $\overline{I}$ the maximum current the ESC is able to provide, we can rewrite Eq. (15) to obtain the maximum propeller acceleration

$$\overline{\dot{\omega}} = \frac{\eta V \overline{I}}{J}\frac{1}{\omega} - \frac{d}{J}\omega^2. \tag{16}$$

Similarly, the minimum acceleration $\underline{\dot{\omega}}$ can be obtained by substituting the minimum current $\underline{I}$ in Eq. (16).

An example of these curves for a typical propeller-ESC combination is shown in Fig. 5 (solid line). While the curves fit the real propeller behavior well at high speeds where the actual physical power expenditure is dominating, at lower speeds the ESC control algorithm and other losses are more apparent:

- Most multirotors are not designed to have their propellers spin backward, which means that the minimum acceleration $\underline{\dot{\omega}}$ should actually go to zero as the speed approaches the minimum $\underline{\omega}$ (usually zero).

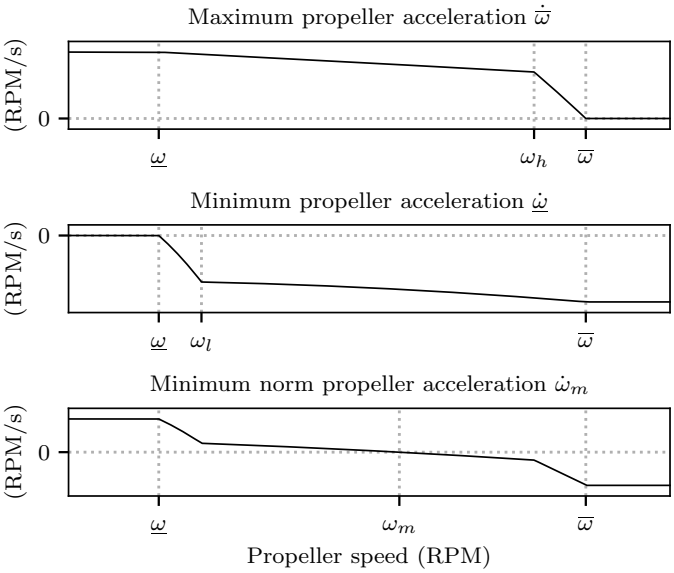

Fig. 6: Proposed propeller limit curves. This approximation allows arbitrarily choosing the equilibrium speed ω_m of the propellers. The first two rows are the maximum $\dot{\overline{\omega}}$ and minimum $\dot{\underline{\omega}}$ acceleration curves as a function of the rotor speed. The last row is the mean $\dot{\omega}_m = \frac{\dot{\overline{\omega}}+\dot{\underline{\omega}}}{2}$.

- As shown in Fig. 5, the maximum acceleration would ideally approach infinity as the propeller speed approaches zero. In practice, however, it is bounded by the internal controller and hardware design of the ESC. As a result, the maximum acceleration should also saturate to $\dot{\overline{\omega}}$ as the speed approaches the minimum $\underline{\omega}$.

We illustrate these modified acceleration limits in Fig. 5 as dashed lines.

B. Propellers' Equilibrium Speed and Proposed Curves

We aim to design propeller acceleration limit curves that not only respect the mechanical constrains in Fig. 5 but also promote speed equalization among propellers. This is achieved by leveraging the fact that the minimum norm solution of the allocation problem in Eq. (12) corresponds to the midpoint between the acceleration limits, i.e. $\dot{\boldsymbol{q}}_\text{n} \to \mathbf{0} \Rightarrow \dot{\omega} \to \frac{\dot{\overline{\omega}}+\dot{\underline{\omega}}}{2}$.

We define the associated speed as the equilibrium speed ω_m

$$\dot{\omega}_m(\omega_m) = \frac{\dot{\overline{\omega}}(\omega_m) + \dot{\underline{\omega}}(\omega_m)}{2} = 0. \tag{17}$$

The key idea is to use this equilibrium speed as a tuning parameter when shaping the maximum and minimum acceleration curves. Since the minimum norm solution $\dot{\omega}_m$ is positive for speeds below ω_m and negative for speeds above ω_m, the allocation passively drives each propeller toward ω_m, removing the need for secondary nullspace goals to balance propeller speeds.

By tuning ω_m, we can prioritize different behaviors: setting it low favors energy efficiency, while choosing a value above nominal hovering speed encourages greater internal force generation and thus faster dynamic response. While we only optimize for energy efficiency in our work, as an overactuated robot, the posture (identified by tilt angles and rotor speeds) affects the manipulability (or *dexterity index* [27]), which is the ability of effectively generating the desired jerk command.

Keeping these requirements in mind, we propose to model the maximum and minimum acceleration curves as composed of two pieces each, as shown in Fig. 6

$$\dot{\overline{\omega}}(\omega) = \begin{cases} c_{0,0}\omega + c_{0,1}\omega^2 + c_{0,2}, & \forall \omega \in [\underline{\omega}, \omega_h] \\ c_{1,0}\omega^2 + c_{1,1}, & \forall \omega \in [\omega_h, \overline{\omega}] \end{cases} \tag{18}$$

$$\dot{\underline{\omega}}(\omega) = \begin{cases} c_{2,0}\omega^2 + c_{2,1}, & \forall \omega \in [\underline{\omega}, \omega_l] \\ c_{3,0}\omega^2 + c_{3,1}, & \forall \omega \in [\omega_l, \overline{\omega}] \end{cases} \tag{19}$$

where ω_h and ω_l are two arbitrary propeller speeds in which we start ramping down the maximum acceleration and ramping up the minimum acceleration, respectively.

With their 9 coefficients, the curves are uniquely identified once the maximum/minimum speeds and accelerations are chosen, together with the equilibrium speed. Recomputing the new limits requires the solution of a small linear system of equations which is detailed in Section A. Then, the new limits can be applied by just recomputing the scaling matrix and bias vector in Eq. (9).

IX. Experimental Evaluation

Here we evaluate the proposed differential allocation methods (ADA, NDA, PDA) against the geometric method GA. The following subsections showcase the effects of the respective enumerated contributions in I-A:

1) Section IX-A compares tracking performance on both slow and fast oscillating trajectories between the geometric allocation (GA), differential allocations that use a weighted pseudoinverse to balance actuator commands

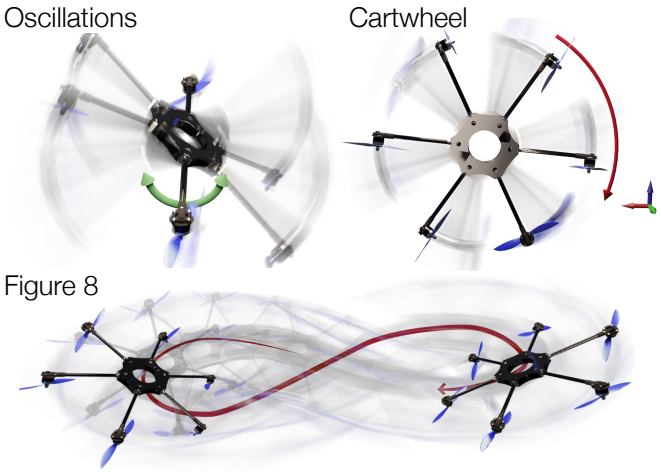

Fig. 7: Illustration of evaluation trajectories. Top Left: Sinusoidal motion around body axes with increasingly faster cycle times, the roll motion is visualized. Top Right: Cartwheel trajectory, in which the robot is commanded to fly upright in the vertical plane and rotate slowly around the perpendicular axis. Bottom: Figure 8 that smoothly excites all axes – similar to the well-known "Lazy 8" in aviation.

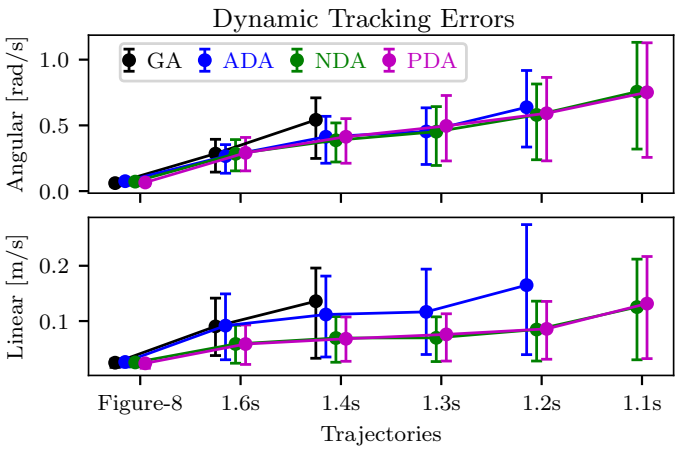

Fig. 8: Mean and quartiles of the linear and angular tracking velocity errors achieved by the geometric (GA) and differential (ADA, NDA, PDA) allocations. When data from an allocation is missing for a specific trajectory, it means that the method failed to complete the trajectory. The use of actuator dynamics allows PDA and NDA to complete all trajectories.

(ADA) and the ones using actuator limit normalization (NDA, PDA).

2) Section IX-B highlights the importance of actuator saturation handling while using differential allocation methods and its influence on power consumption, particularly comparing the use of nullspace commands NDA and propeller curves PDA, vs. the total absence of any propeller speed regulation techniques NDAnN.
3) Section IX-C evaluates behaviors while flying through allocation singularities, showing the robustness of differential over geometric allocations.
4) Section IX-D demonstrates how propeller curves (PDA) can smoothly turn off propellers in flight and use the corresponding arms as proper manipulators.

A. Trajectory tracking

We fly the robot through seven different trajectories: a slow Figure-8 and six increasingly fast and dynamic trajectories that involve different rotations on the spot (Fig. 7). In Table I we summarize the success of each allocation method in completing the desired trajectories, eventually bringing all of them to failure. A run is considered successful as long as the controller does not diverge, i.e. the errors are bounded. We also show their dynamic tracking errors (linear and angular) in Fig. 8. Here we focus specifically on dynamic tracking errors as we are interested in the dynamic performance of these allocation methods, but a full statistical overview of the tracking errors (including position and attitude) is available in Fig. 14 later in the Appendix.

The first to fail is GA, which completes the Figure-8 and only the two slowest oscillating trajectories. This was expected as this method assumes instantaneous control of the actuators, without any knowledge of actuator dynamics and limits.

The second method to fail is ADA, which completes most of the proposed trajectories. Despite not having knowledge of the exact actuator dynamics and limits, this method still performs comparably well, although it is highly dependent on a careful tuning of the weight matrix $\boldsymbol{W}$ to balance the usage of the actuators.

	Fig-8	1.6 s	1.4 s	1.3 s	1.2 s	1.1 s	1.0 s
GA							
ADA							
NDA							
PDA							
Ang. vel.	0.5	2.3	2.8	3.2	3.5	4.0	-
Lin. vel.	0.4	0.17	0.20	0.25	0.35	0.6	-

TABLE I: The table summarizes whether an allocation method was successful in completing the desired trajectory (green) or not (red). Allocation methods are on the rows while the trajectories are on the columns slowest (left) to fastest (right). The best methods (PDA and NDA) require the use of actuator dynamics and normalized actuator limits. The last two rows show the maximum linear $(\mathrm{m/s})$ and angular $(\mathrm{rad/s})$ body velocity norms reached during the corresponding trajectory.

Once the allocation problem is normalized—by incorporating actuator dynamics and limits and removing the weight matrix $\boldsymbol{W}$—both NDA (with nullspace objectives) and PDA (using propeller power curves) demonstrate the highest tracking performance. Notably, PDA achieves this while consistently keeping the nullspace available during nominal flight. This leads to two key benefits: first, it eliminates the need for task-switching or prioritization strategies when transitioning from free flight to manipulation tasks (Section IX-D), since additional objectives naturally exploit the already free nullspace; second, it avoids the need for in-flight tuning of the nullspace goal, as the power curves are inherently derived from the physical characteristics of the rotors.

To assess whether the tracking errors between the two methods differ significantly, we perform Welch's unequal variances t-test. We set the null hypothesis that the velocity error distributions from Fig. 8 have equal mean values, rejecting it if the p-value is below 0.05. The results indicate no statistically significant difference between the two approaches, with p-values of 0.106 for linear and 0.788 for angular tracking errors.

In summary, PDA offers robust tracking performance comparable to NDA, while providing practical advantages through nullspace availability and physically grounded design.

B. Balancing propellers' speed and power consumption

During flight, the speed of the propellers needs to be carefully regulated to avoid propeller saturation and unnecessarily high power consumption. While the geometric allocation (GA) by definition always minimizes the propeller speed to obtain the desired control wrench, differential methods (ADA, NDA, PDA) only operate in terms of propeller accelerations to generate the desired jerk. In our experiments, we try to keep the propellers close to the $5800RPM$ hovering speed of our system, either through nullspace objectives (ADA, NDA) or the proposed propeller curves (PDA).

If neither of these are present, the speeds can slowly drift towards the propeller upper limit of $8800RPM$. To demon-

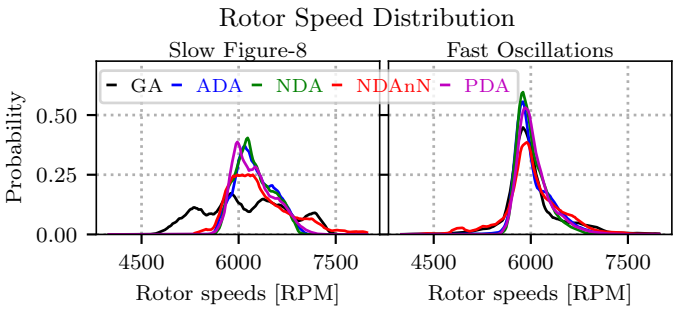

Fig. 9: Comparison of rotor speeds distributions between the different allocation methods. When the secondary objective is not present (NDAnN) or in case of the geometric allocation (GA), the rotor speeds distribution is wider and the propellers are pushed closer to their limits.

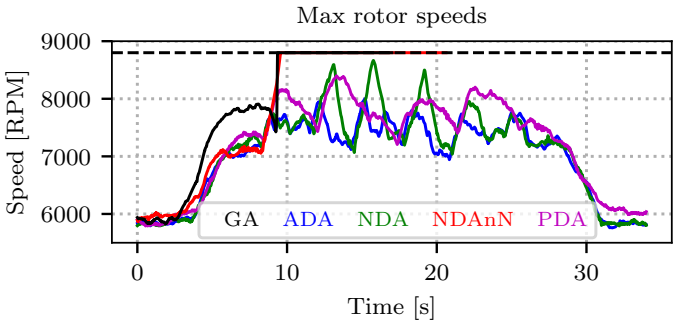

Fig. 11: Maximum rotor speeds experienced during the cartwheel trajectory by the different methods. The speed limit allowed by the ESCs is 8800 RPM (dashed line). GA and NDAnN both hit the saturation and diverge, while the other methods keep the propellers away from the limit.

strate this, in Fig. 9 we compare the rotor speeds distributions between the different allocation methods. We additionally evaluate one of the best methods, NDA, but without the nullspace secondary objective that regulates propeller speeds, obtaining NDAnN. We can see NDAnN's speed distribution having an upper tail towards higher rotor speeds during the slow Figure-8 trajectory, while the other differential methods manage to keep the speeds close to $5800RPM$. GA also has a wide speed distribution as it regulates in terms of *energy* efficiency, without regard for actuator limits.

If we look at the propellers' power consumption in Fig. 10, the three differential methods perform very similarly as they possess similar speed distributions. On the other hand, NDAnN slowly pushes the propeller speed up, unnecessarily increasing the power consumption up to 18% more. The geometric approach is, as expected, the most energy efficient.

C. Flying through singularities

As mentioned in Section IV, geometric allocation methods suffer from singularities which become particularly apparent

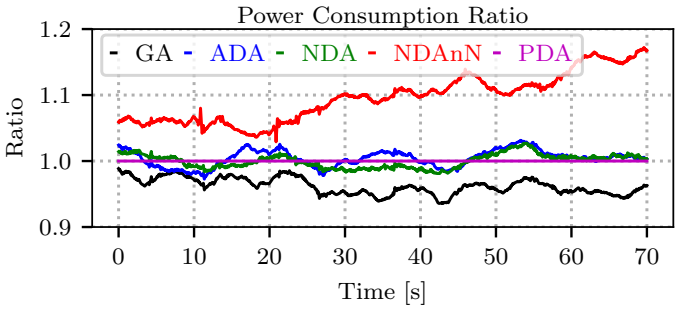

Fig. 10: Power usage ratio between PDA and the other allocation methods during the slow Figure-8 trajectory. GA is up to 6% more efficient than the other differential methods, which have similar consumptions. NDAnN increases the average propeller speeds over time, leading to a power consumption up to 18% higher than the PDA method. The power consumption is computed using Eq. (15) with $J = 4.5e^{-4}Kgm^2$ and $d = 3.5e^{-7}Nms^2$. The servomotors power consumption is two orders of magnitude smaller and thus negligible.

when the robot is close to a 90° pitch or roll angle [21]. Here we show that the proposed differential allocation methods are not affected by these singularities and can still track the desired trajectory even when the robot is at very high angles, unlocking full omnidirectionality. To further complicate things, in these configurations, the weight of the robot is often compensated by only few propellers, pushing them close to their speed limits.

Consider the cartwheel trajectory in Fig. 7, where the robot starts in horizontal hover, then pitches up to 90° and starts rotating around its body z axis, resembling a rotating cartwheel in the air. GA diverges early as it is not able to handle the singularity. NDAnN also quickly saturates the propeller speeds and diverges due to its lack of nullspace goals and limit curves. The other differential methods (ADA, NDA, PDA) all successfully complete the trajectory.

All differential allocations try to keep the propellers' speeds low, ADA and NDA using a secondary objective to do so and PDA using propeller limit curves, see Fig. 11.

D. Stopping propellers for manipulation

The possibility of turning off propellers at will opens the door to: *(i)* a safer interaction with the environment, since the propellers close to a surface can be just disabled; *(ii)* additional manipulation capabilities, as the rotating arms can be used for active physical interaction.

Here, we qualitatively show how to use PDA and the propeller's limit curves to stop one propeller in-flight, summarized in Fig. 12. Meanwhile, we use the nullspace of the allocation to rotate the corresponding arm and screw a bolt into a wall.

In Fig. 13, we present the actuator states and rate commands over time during the manipulation experiment, highlighting the moment when the propeller stops and the arm rotates while approaching the wall.

Note the change in desired propeller acceleration, which is used to stop the propeller, as well as the controlled angular velocity of the arm–both during free flight and throughout the screwing interaction.

We enable this behavior by using the limit curves in Fig. 6 to temporarily impose a negative maximum acceleration on

Fig. 12: From left to right: the Aerial Robot is hovering in place with the interaction propeller turned off. Then the robot approaches the board and starts rotating the arm to screw the bolt in. Finally, the propeller is re-enabled and the arm aligns with gravity again to balance the actuators' usage. The stopped propeller is highlighted in yellow.

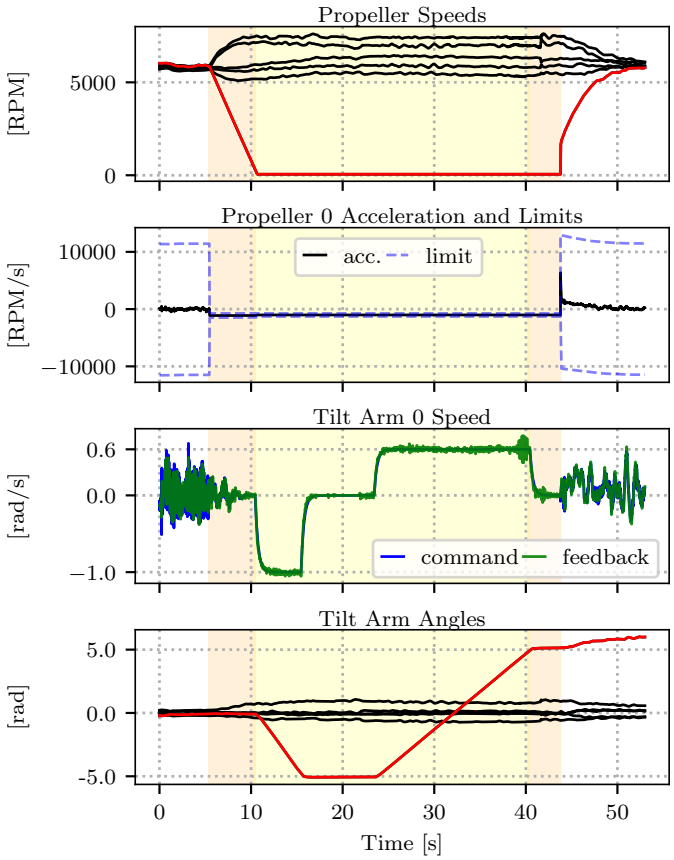

Fig. 13: Top to bottom: the speeds of the six propellers (in red the one turned off), the propeller acceleration (and its limits as dashed lines), the speed of the interaction arm, and last the angles of the six arms (in red the manipulation arm). When the acceleration limits are both negative, the propeller slows down until it stops. In the orange shading, the propeller is turned off without explicitly commanding the relative tilt angle. In the yellow shading, the tilt angle is commanded a speed between -1 and $0.6\,\mathrm{rad/s}$ through the nullspace. In the end, the arm is released and the allocation naturally brings back the propeller to hovering speed to balance the propellers' usage.

the propeller, forcing the allocation to decelerate it until it comes to a stop. At the end of the interaction, we lift these limits by reapplying the original curves, causing the allocation to naturally restore the balance between propeller speeds.

To track the desired arm speed $\dot{\alpha}^*$ in case of unknown friction in the robot's mechanical assembly or with the environment, we use an integral control action in the secondary allocation command

$$\dot{q}^* = \dot{\alpha}^* + k_I \int (\dot{\alpha}^* - \dot{\alpha})\, dt, \tag{20}$$

where $k_I \in \mathbb{R}$ is the integrator gain and $\dot{\alpha}$ the arm rotation velocity feedback from the servomotors.

X. Conclusion

In this work, we extend the state-of-the-art differential allocation to include actuator power dynamics and limits.

Experimental results across diverse maneuvers, including a screwing manipulation task, demonstrate improved tracking of high-dynamic trajectories and seamless stopping and restarting of propellers, while keeping the nullspace free to exploit the over-actuation of the system. Additionally, we provide statistical proof that our nullspace-free formulation does not have any performance drawbacks over classical nullspace-based methods.

We introduce propeller limit curves as a compact representation of actuator acceleration limits, enabling a unified treatment of actuator dynamics and user-defined preferences.

The method leverages actuator feedback but avoids reliance on acceleration feedback, which is often unreliable on aerial robots where propeller noise on the IMU and system dynamics are much closer in frequency than on e.g. racing drones. However, our method requires an accurate system model for wrench generation.

Current limitations include the absence of inter-rotor airflow modeling and the assumption of first-order actuator dynamics. Extending the framework to capture these effects is a promising direction for future work.

The proposed formulation is fully compatible with existing wrench-based control architectures and can be integrated without modifying the high-level control stack. It provides a practical and scalable improvement over geometric and differential allocation methods, advancing the capabilities of aerial robotic systems.

Appendix
Computing the Propeller Limit Curves

The 9 coefficients of the power curves can be computed by solving the following linear system of equations

$$\overline{\dot{\omega}}(\underline{\omega}) = c_{0,0}\underline{\omega} + c_{0,1}\underline{\omega}^2 + c_{0,2}, \quad (21a)$$
$$c_{0,0}\omega_h + c_{0,1}\omega_h^2 + c_{0,2} = c_{1,0}\omega_h^2 + c_{1,1}, \quad (21b)$$
$$0 = c_{1,0}\overline{\omega}^2 + c_{1,1}, \quad (21c)$$
$$\dot{\omega}_1 = c_{1,0}\omega_h^2 + c_{1,1}, \quad (21d)$$
$$0 = c_{2,0}\underline{\omega}^2 + c_{2,1}, \quad (21e)$$
$$c_{2,0}\omega_l^2 + c_{2,1} = c_{3,0}\omega_l^2 + c_{3,1}, \quad (21f)$$
$$\dot{\omega}_2 = c_{2,0}\omega_l^2 + c_{2,1}, \quad (21g)$$
$$\underline{\dot{\omega}}(\overline{\omega}) = c_{3,0}\overline{\omega}^2 + c_{3,1}, \quad (21h)$$
$$0 = c_{0,0}\omega_m + c_{0,1}\omega_m^2 + c_{0,2} + c_{3,0}\omega_m\omega_m^2 + c_{3,1}. \quad (21i)$$

This system's solution is unique for the coefficients $c_{i,j}$, given the minimum and maximum propeller speeds $\underline{\omega}$, $\overline{\omega}$, and accelerations $\overline{\dot{\omega}}(\underline{\omega})$, $\underline{\dot{\omega}}(\overline{\omega})$, the speed at which the quadratic drag effect becomes predominant ω_h and its acceleration $\overline{\dot{\omega}}(\omega_h)$, the speed where we start saturating the propeller acceleration to avoid reducing its speed below ω_l and its acceleration $\underline{\dot{\omega}}(\omega_l)$, and the desired average propeller speed ω_m.

In our experiments, we have $\omega_m = 5800RPM$, while $\underline{\omega} = 0$ and $\overline{\omega} = 8700RPM$. In our experience, the propeller's acceleration starts noticeably decreasing around $\sim 8000RPM$. Thus we use $\omega_h = 7800RPM$, the 90% of the total speed range. The lower limit ω_l is a limitation of the ESC, as the speed cannot go below 0. For symmetry, we choose $900RPM$, or the 10% of the total speed range. The maximum $\overline{\dot{\omega}}(\omega_h)$ and minimum $\underline{\dot{\omega}}(\omega_l)$ accelerations in those points are 80% of the absolute maximum and minimum acceleration limits $\overline{\dot{\omega}}(\underline{\omega})$, $\underline{\dot{\omega}}(\overline{\omega})$. Based on our ESC's limitations, we have $\overline{\dot{\omega}}(\underline{\omega}) = 12000RPM/s$ and $\underline{\dot{\omega}}(\overline{\omega}) = -14000RPM/s$.

We can then compute the coefficients $c_{i,j}$ from (21) with a small matrix inversion. The values of minimum and maximum acceleration can be changed arbitrarily: both could be negative to stop the propellers in-flight, or restored to their original values to turn them on again.

REFERENCES

[1] A. Ollero, M. Tognon, A. Suarez, D. Lee, and A. Franchi, "Past, present, and future of aerial robotic manipulators," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 626–645, 2022.

[2] K. Bodie, M. Brunner, M. Pantic, S. Walser, P. Pfändler, U. Angst, R. Siegwart, and J. Nieto, "Active interaction force control for contact-based inspection with a fully actuated aerial vehicle," *IEEE Transactions on Robotics*, vol. 37, no. 3, pp. 709–722, 2021.

[3] S. Park, J. Lee, J. Ahn, M. Kim, J. Her, G.-H. Yang, and D. Lee, "Odar: Aerial manipulation platform enabling omnidirectional wrench generation," *IEEE/ASME Transactions on Mechatronics*, vol. 23, no. 4, pp. 1907–1918, 2018.

[4] M. Tognon, H. A. T. Chávez, E. Gasparin, Q. Sablé, D. Bicego, A. Mallet, M. Lany, G. Santi, B. Revaz, J. Cortés, and A. Franchi, "A truly-redundant aerial manipulator system with application to push-and-slide inspection in industrial plants," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1846–1851, 2019.

[5] M. Ryll, G. Muscio, F. Pierri, E. Cataldi, G. Antonelli, F. Caccavale, D. Bicego, and A. Franchi, "6d interaction control with aerial robots: The flying end-effector paradigm," *The International Journal of Robotics Research*, vol. 38, no. 9, pp. 1045–1062, 2019.

[6] D. Brescianini and R. D'Andrea, "Design, modeling and control of an omni-directional aerial vehicle," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016.

[7] E. Cuniato, C. Geckeler, M. Brunner, D. Strübin, E. Bähler, F. Ospelt, M. Tognon, S. Mintchev, and R. Siegwart, "Design and control of a micro overactuated aerial robot with an origami delta manipulator," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 5352–5358.

[8] M. Allenspach, K. Bodie, M. Brunner, L. Rinsoz, Z. Taylor, M. Kamel, R. Siegwart, and J. Nieto, "Design and optimal control of a tiltrotor micro-aerial vehicle for efficient omnidirectional flight," *International Journal of Robotics Research*, vol. 39, no. 10-11, pp. 1305–1325, 2020.

[9] M. Ryll, D. Bicego, M. Giurato, M. Lovera, and A. Franchi, "Fast-hex - A morphing hexarotor: Design, mechanical implementation, control and experimental validation," *CoRR*, vol. abs/2004.06612, 2020.

[10] J. T. M. W. Mueller, J. Tang, and M. W. Mueller, "Pairtilt: Design and control of an active tilt-rotor quad-copter for improved efficiency and agility."

[11] M. Brunner, G. Rizzi, M. Studiger, R. Siegwart, and M. Tognon, "A planning-and-control framework for aerial manipulation of articulated objects," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10 689–10 696, 2022.

[12] M. Ryll, D. Bicego, and A. Franchi, "A truly redundant aerial manipulator exploiting a multi-directional thrust base," *IFAC-PapersOnLine*, vol. 51, no. 22, pp. 138–143, 2018, 12th IFAC Symposium on Robot Control SYROCO 2018.

[13] M. Zhao, K. Okada, and M. Inaba, "Versatile articulated aerial robot dragon: Aerial manipulation and grasping by vectorable thrust control," *The International Journal of Robotics Research*, vol. 42, no. 4-5, pp. 214–248, 2023.

[14] T. Nishio, M. Zhao, K. Okada, and M. Inaba, "Design, control, and motion-planning for a root-perching rotor-distributed manipulator," *IEEE Transactions on Robotics*, 2023.

[15] M. Ryll, H. H. Bülthoff, and P. R. Giordano, "A novel overactuated quadrotor unmanned aerial vehicle: Modeling, control, and experimental validation," *IEEE Transactions on Control Systems Technology*, vol. 23, no. 2, pp. 540–556, 2015.

[16] R. Falconi and C. Melchiorri, "Dynamic model and control of an over-actuated quadrotor uav," *IFAC Proceedings Volumes*, vol. 45, no. 22, pp. 192–197, 2012, 10th IFAC Symposium on Robot Control.

[17] Z. Qin, J. Wei, M. Cao, B. Chen, K. Li, and K. Liu, "Design and flight control of a novel tilt-rotor octocopter using passive hinges," *IEEE Robotics and Automation Letters*, 2023.

[18] S. Rajappa, M. Ryll, H. H. Bülthoff, and A. Franchi, "Modeling, control and design optimization for a fully-actuated hexarotor aerial vehicle with tilted propellers," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 4006–4013.

[19] M. Ryll, D. Bicego, and A. Franchi, "Modeling and control of fast-hex: A fully-actuated by synchronized-tilting hexarotor," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 1689–1694.

[20] F. Morbidi, D. Bicego, M. Ryll, and A. Franchi, "Energy-efficient trajectory generation for a hexarotor with dual- tilting propellers," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 6226–6232.

[21] K. Bodie, Z. Taylor, M. Kamel, and R. Siegwart, "Towards efficient full pose omnidirectionality with overactuated MAVs," in *Springer Proceedings in Advanced Robotics*, 2020, pp. 85–95.

[22] T. A. Johansen and T. I. Fossen, "Control allocation—a survey," *Automatica*, vol. 49, no. 5, pp. 1087–1103, 2013.

[23] J. Sugihara, M. Zhao, T. Nishio, K. Okada, and M. Inaba, "Beatle—self-reconfigurable aerial robot: Design, control and experimental validation," *IEEE/ASME Transactions on Mechatronics*, 2024.

[24] S. Sun, A. Romero, P. Foehn, E. Kaufmann, and D. Scaramuzza, "A comparative study of nonlinear mpc and differential-flatness-based control for quadrotor agile flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3357–3373, 2022.

[25] G. Notomista, S. Mayya, M. Selvaggio, M. Santos, and C. Secchi, "A set-theoretic approach to multi-task execution and prioritization," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 9873–9879.

[26] C. Lanegger, M. Pantic, R. Bähnemann, R. Siegwart, and L. Ott, "Chasing millimeters: design, navigation and state estimation for precise in-flight marking on ceilings," *Autonomous Robots*, vol. 47, no. 8, pp. 1405–1418, 2023.

[27] B. Siciliano, O. Khatib, and T. Kröger, *Springer handbook of robotics*. Springer, 2008, vol. 200.

[28] A. D. Ames, G. Notomista, Y. Wardi, and M. Egerstedt, "Integral control barrier functions for dynamically defined control laws," *IEEE Control Systems Letters*, vol. 5, no. 3, pp. 887–892, 2020.

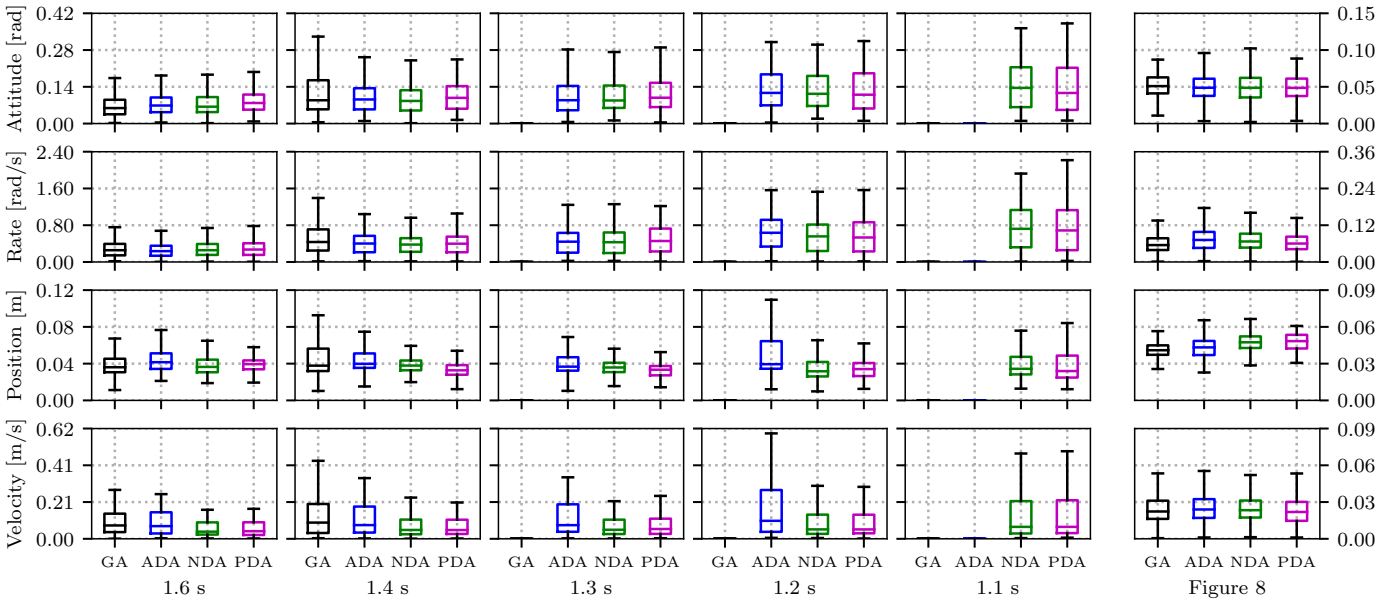

Fig. 14: Tracking results from the different allocation methods on several test trajectories. From top to bottom row: Attitude, Angular Velocity, Position and Linear Velocity errors. The five columns on the left show the results with increasingly fast sinusoidal trajectories, from a period of 1.6 s to 1.1 s. The additional column on the right shows the results with a slow Figure-8 trajectory. If no data is present, the corresponding allocation method diverged while following the trajectory.

[29] E. Cuniato, N. Lawrance, M. Tognon, and R. Siegwart, "Power-based safety layer for aerial vehicles in physical interaction using lyapunov exponents," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6774–6781, 2022.

[30] F. Flacco, A. De Luca, and O. Khatib, "Control of redundant robots under hard joint constraints: Saturation in the null space," *IEEE Transactions on Robotics*, vol. 31, no. 3, pp. 637–654, 2015.

Eugenio Cuniato is currently a Postdoc at the Autonomous Systems Lab, ETH Zurich, Switzerland. His research focus is on control of aerial manipulators for high-performance aerial physical interaction.

Mike Allenspach received his Master's degree in Robotics, Systems, and Control (2020) and his Ph.D. in Robotics (2025) from ETH Zurich, Switzerland. His research focuses on planning and control for aerial manipulation.

Thomas Stastny is currently a Senior Researcher in the Autonomous Systems Lab (ASL) at ETH Zurich focusing on learning-based perception and control for omnidirectional aerial manipulators.

Helen Oleynikova is also a Senior Researcher in the Autonomous Systems Lab (ASL) at ETH Zurich focusing on volumetric mapping and planning for both manipulation and flying robots.

Roland Siegwart is professor for autonomous mobile robots at ETH Zurich, founding co-director of the technology transfer center Wyss Zurich and board member of multiple high tech companies. He was professor at EPFL Lausanne (1996 – 2006), held visiting positions at Stanford University and NASA Ames and was Vice President of ETH Zurich (2010-2014). His interests are in the design, control and navigation of flying, wheeled and walking robots.

Michael Pantic is a Senior Researcher at the Autonomous Systems Lab (ASL) at ETH Zurich, where he is leading the research on aerial robots. In his own research, he focuses on system architecture, perception and navigation for aerial robots.